

Manual da Aula: Persistência de Dados em Arquivo JSON

Módulo de escrita síncrona com manipulação de sistema de arquivos e serialização

Unidade Curricular: Programação Back-End (JS / Node.js)

Carga Horária: 5 aulas de 45 minutos (ou planejado)

Eixo Tecnológico: Informação e Comunicação

Capacidades Técnicas: Persistência local; estruturação de coleções JSON; uso do módulo nativo fs.

Capacidades Socioemocionais: Pensamento analítico (fluxo memória-disco); autogestão e sintaxe rigorosa.

2. Fundamentação Conceitual

Volatilidade da Memória RAM

Variáveis e objetos existem na memória temporária. Quando o script finaliza a execução, os dados são descartados.

Persistência em Disco

Gravação de estruturas em suporte duradouro (HD/SSD) para garantir disponibilidade em execuções futuras.

Módulo Nativo fs (File System)

Módulo embutido no Node.js. O método `fs.writeFileSync(caminho, texto)` grava de forma síncrona sobrescrevendo ou criando o arquivo.

Serialização com JSON.stringify

Discos armazenam texto bruto ou binário. `JSON.stringify(dado, null, 2)` formata objetos em strings legíveis com indentação.

3. Exemplo Guiado para Digitação (Prática em Conjunto)

Cenário: Registro de máquinas operacionais e exportação física para o arquivo `maquinas.json`.

1. Abra o VS Code na pasta da aula e crie o arquivo `exemplo_gravacao.js`.
2. Digite e inspecione o código abaixo, salvando em seguida.
3. Execute no terminal com: `node exemplo_gravacao.js`.

```
JavaScript (Node.js) exemplo_gravacao.js

// Importa o módulo nativo do Node.js para manipulação de arquivos
const fs = require('fs');

console.log("=== SISTEMA DE PERSISTÊNCIA: REGISTRO DE MÁQUINAS ===");

// 1. Definição da estrutura de dados em memória (Array de Objetos)
const maquinasIndustriais = [
  { id: 101, nome: "Torno Mecânico Universal", setor: "Usinagem", operacional: true },
  { id: 102, nome: "Fresadora Ferramenteira", setor: "Usinagem", operacional: false },
  { id: 103, nome: "Prensa Hidráulica 50T", setor: "Estampagem", operacional: true }
];

// 2. Conversão da estrutura em formato texto legível (JSON indentado)
const dadosParaGravar = JSON.stringify(maquinasIndustriais, null, 2);

// 3. Gravação física no disco rígido
const nomeDoArquivo = "maquinas.json";
fs.writeFileSync(nomeDoArquivo, dadosParaGravar);

console.log(`\nGravação concluída com sucesso.`);
console.log(`Verifique o arquivo '${nomeDoArquivo}' gerado na barra lateral do VS Code.`);
```

4. Exercícios de Fixação

Aplique os conceitos de estruturação de coleções de objetos, serialização com `JSON.stringify` e gravação síncrona com `fs.writeFileSync`.

Exercício 1: Registro de Sensores Industriais

Contexto: Uma célula de manufatura automatizada precisa registrar os parâmetros de 3 sensores de temperatura e pressão instalados em um reator.

Requisitos:

- Criar arquivo `exercicio1_sensores.js`.
- Definir array com 3 objetos contendo: `codigo` (inteiro), `tipo` ("Temperatura" / "Pressão"), `leituraAtual` (decimal) e `status` ("Operando" / "Alerta").
- Converter com indentação de 2 espaços e gravar fisicamente em `sensores.json`.
- Exibir mensagem de sucesso no terminal ao finalizar.

Exercício 2: Cadastro de Ferramental com Entrada Dinâmica

Contexto: A ferramentaria precisa registrar ferramentas manuais interativamente pelo terminal e consolidar o lote no disco.

Requisitos:

- Criar arquivo `exercicio2_ferramentas.js` e utilizar a biblioteca `readline-sync`.
- Perguntar quantas ferramentas serão registradas e iterar via `laço for`.
- Para cada ferramenta, solicitar: `nome` (string), `quantidade` (inteiro) e `custoUnitario` (float), inserindo no array via `.push()`.
- Persistir os dados em `ferramentas.json` e exibir confirmação com contagem de itens.

Exercício 3: Histórico de Inspeção de Qualidade com Validação de Lote

Contexto: Um inspetor de qualidade afere a espessura de 4 chapas de aço. O sistema compila as medições e valida a tolerância técnica do lote.

Requisitos:

- Criar arquivo `exercicio3_inspecao.js`.
- Estruturar o objeto `relatorioInspecao` com: `data` ("2026-09-23"), `inspetor` (nome), `amostras` (array de 4 números em mm) e `loteAprovado` (booleano).
- O lote só é aprovado se todas as medidas forem ≥ 12.0 mm.
- Converter para JSON e salvar no arquivo `inspecao_qualidade.json`, emitindo veredito no console.

5. Gabaritos dos Exercícios de Fixação (Parte 1)

Gabarito – Exercício 1: Sensores Industriais

```
JavaScript (Node.js) exercicio1_sensores.js

const fs = require('fs');

const sensores = [
  { codigo: 1001, tipo: "Temperatura", leituraAtual: 85.4, status: "Operando" },
  { codigo: 1002, tipo: "Pressão", leituraAtual: 6.2, status: "Operando" },
  { codigo: 1003, tipo: "Temperatura", leituraAtual: 102.7, status: "Alerta" }
];

const dadosJSON = JSON.stringify(sensores, null, 2);
fs.writeFileSync('sensores.json', dadosJSON);

console.log("Arquivo 'sensores.json' gerado com sucesso.");
```

Gabarito – Exercício 2: Ferramental com Entrada Dinâmica

```
JavaScript (Node.js) exercicio2_ferramentas.js

const fs = require('fs');
const entrada = require('readline-sync');

console.log("=== CADASTRO DE FERRAMENTAL ===");
const totalItens = entrada.questionInt("Quantas ferramentas deseja cadastrar? ");
const listaFerramentas = [];

for (let i = 0; i < totalItens; i++) {
  console.log(`\nItem ${i + 1} de ${totalItens}:`);
  const nome = entrada.question("Nome da ferramenta: ");
  const quantidade = entrada.questionInt("Quantidade: ");
  const custoUnitario = entrada.questionFloat("Custo unitario (R$): ");

  listaFerramentas.push({
    nome: nome,
    quantidade: quantidade,
    custoUnitario: custoUnitario
  });
}

fs.writeFileSync('ferramentas.json', JSON.stringify(listaFerramentas, null, 2));

console.log("\n-----");
console.log(`Sucesso: ${listaFerramentas.length} itens gravados em 'ferramentas.json'.`);
console.log("-----");
```

5. Gabaritos dos Exercícios de Fixação (Parte 2)

Gabarito — Exercício 3: Inspeção de Qualidade e Validação

```
JavaScript (Node.js) exercicio3_inspecao.js

const fs = require('fs');

const amostrasColetadas = [12.1, 12.3, 11.9, 12.0];

// Verifica se todas as amostras atendem ao critério mínimo de 12.0 mm
let aprovado = true;
for (let i = 0; i < amostrasColetadas.length; i++) {
  if (amostrasColetadas[i] < 12.0) {
    aprovado = false;
    break;
  }
}

const relatorioInspecao = {
  data: "2026-09-23",
  inspetor: "Carlos Eduardo",
  amostras: amostrasColetadas,
  loteAprovado: aprovado
};

fs.writeFileSync('inspecao_qualidade.json', JSON.stringify(relatorioInspecao, null, 2));

console.log("=== RELATÓRIO DE QUALIDADE GERADO ===");
console.log(`Status do Lote: ${aprovado ? "APROVADO" : "REPROVADO"}`);
console.log("Arquivo 'inspecao_qualidade.json' gravado em disco.");
```

Critérios de Avaliação e Observabilidade Docente:

- Conferência da correta importação e ausência de dependências externas para o módulo nativo fs.
- Compreensão do terceiro parâmetro de indentação em JSON.stringify(..., null, 2).
- Checagem da integridade sintática e validação física dos arquivos JSON gerados na raiz do projeto.